

The Agent PR Review Checklist

The checks that catch bugs the tests pass clean.

When a fleet of AI agents writes your pull requests, the tests going green means almost nothing. Agents produce code that compiles, passes the tests they also wrote, and still ships a shell-injection vulnerability. This is the exact checklist I run against every agent-generated PR before it merges — security first, because that's the failure mode automated checks miss most. Use it as a gate: if you can't tick a box, the PR doesn't merge.

1 · Security — run this pass first, every time

Agents are trained on public code, and public code is full of insecure patterns. These are the classes I actually catch.

- ☐ **Command / shell injection.** Every exec/spawn/child_process is checked for interpolated input — args as an array, never a concatenated command string.
- ☐ **Path traversal.** Any path built from request input is resolved and confined to an allowed root; `../` can't escape.
- ☐ **SQL / NoSQL injection.** Queries parameterized; no string-built SQL, no unvalidated object spread into a Mongo filter.
- ☐ **Secrets.** No hardcoded keys or connection strings; nothing secret is logged or committed; new env vars are placeholders in `.env.example` only.
- ☐ **AuthN / AuthZ.** Every new endpoint checks who the caller is and whether they may act on this specific resource. Watch for IDOR.
- ☐ **SSRF & outbound requests.** Server-side URLs from user input are allowlisted; no fetching arbitrary internal addresses.
- ☐ **Dynamic execution.** No eval, Function(), dynamic require, or unsafe deserialization on untrusted input.
- ☐ **Dependencies.** Every new package is intentional, maintained, license-clean; run npm/pip audit. A new dep is new attack surface.
- ☐ **Output encoding.** User-controlled data rendered to HTML is escaped; no dangerouslySetInnerHTML on untrusted input.

2 · Correctness the tests won't catch

Agents write both the code and the tests — so the tests often encode the same misunderstanding.

- ☐ **The tests assert behavior**, not implementation. A test that mocks the thing under test is theatre.
- ☐ **Error paths are handled**, not swallowed. No empty catch, no unhandled rejections. Failures surface.
- ☐ **Edge cases**: empty, zero, negative, null/undefined, empty array, huge input, unicode. Agents optimize the happy path.
- ☐ **Boundaries & off-by-one** on every loop, slice, pagination, or range.
- ☐ **Concurrency / races** where two requests hit the same state — no unguarded read-modify-write.
- ☐ **The change matches the ticket**. Agents over-reach: unrelated refactors and renames hide the real change.

3 · Data & irreversible operations

- ☐ **Migrations are reversible** and don't silently drop or truncate data. Destructive ops are reviewed twice.
- ☐ **Transaction boundaries** wrap multi-step writes so a partial failure can't corrupt state.
- ☐ **No production data** in fixtures, logs, or test files.

4 · The failures specific to agents

These don't show up in a normal human-PR review because humans don't make them. Agents do, consistently.

- ☐ **Hallucinated APIs**. Methods, flags, or config keys that don't exist on the real library version.
- ☐ **Confidently wrong comments**. The comment describes the intent; the code does something else.
- ☐ **Plausible-but-wrong constants** — a timeout or limit that looks reasonable and is wrong for your system.
- ☐ **Copy-pasted patterns** that don't fit your conventions, error handling, or logging.
- ☐ **Tests that assert the bug**. The most dangerous output: buggy code plus a test that locks it in as correct.

Run passes 1 → 4 in order — security first. Don't merge on green; merge on a completed checklist.

This is the adversarial review I run on every PR in a Delivery Sprint: an agent fleet ships the volume, every pull request passes this gate before the client sees it. Speed without the risk.

Want your backlog shipped this way — reviewed before it reaches you? Book a 30-minute scoping call at adrian-rusan.com/services